Lab: Coding up a contact form

- Build a contact form
- Access form values with JavaScript
- Check form values and show user alerts
- Modify page to show loading icon / result / error
- Optional: Submit form values to the server and deploy on brighton.domains

---

Preview of what we are building:

Try out a working version of the contact form here.

How the contact form will work

Our contact form will consist of four separate sections:

1. the form itself
2. a loading screen, shown while data is transmitted to the server
3. a success screen, shown when the data was successfully submitted
4. an error screen, shown when something went wrong

Only one of these sections is visible at any given time!

We control the visibility of sections with the CSS **`display`** property:

- **`display: none;`** - section is invisible
- **`display: block;`** - section is visible

Initially, only the form section is visible, while all other sections are hidden.

Step 1: Set up folder and files

- Set up a new folder for this example. Mine is called **`js3`**, but you call it whatever you like

- Copy over **`index.html, index.css, index.js`** from last week's lab, or create new files if you like. The HTML, CSS and JS should be readily linked up.

- Your starting HTML should look like this:

```
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width">
    <link href="index.css" rel="stylesheet" type="text/css" />
    <script src="index.js"></script>
    <title>Contact form</title>
</head>
<body>



</body>
</html>
```

- Your starting CSS should look like this:

```
body {
    font-family: sans-serif;
}
```

- Your starting JS should look like this:

```
window.addEventListener("load", function(){


});
```

- Save all changes, then open **index.html** in your browser.
- Arrange your workspace so that the browser and editor windows are next to each other.

Step 2: Create the HTML page and CSS styles

- Edit **index.html** to create the basic page structure with the four sections described above. Inside the **<body>** element, add four ** ** elements and give them appropriate IDs:

```
<body>
    <div id="contact">
    </div>
    <div id="loading">
    </div>
    <div id="success">
    </div>
    <div id="error">
    </div>
</body>
```

- In the **contact** section, add a **<form>** element. We will use a **<fieldset>** inside the form so that the browser draws a border around the form and shows a caption (the **<legend>**)

```
<div id="contact">
    <form>
        <fieldset>
            <legend>Leave a message and we'll contact you shortly!</legend>

        </fieldset>
    </form>
</div>
```

- Note that our form does not have any attributes like **action** or **method** - we don't need them because we'll handle the form submit ourselves in JavaScript.

- The form has no **<input>** fields yet, let's add them next. We want fields for an email address, a subject line, and a message. Each of them has a **<label>** for accessibility. We also have a **submit** button to submit the form. All of them are inside our **<fieldset>**:

```
<fieldset>

    <legend>Leave a message and we'll contact you shortly!</legend>

    <label for="email">Email*</label>
```

```
    <input id="email" type="email" />

    <label for="subject">Subject*</label>
    <input id="subject" type="text" />

    <label for="message">Message*</label>
    <textarea id="message"></textarea>

    <input type="submit" value="Send Message" />

</fieldset>
```

- Notice the stars (*) in the label text indicating that these fields are mandatory. When the form is submitted, we will check these values in JavaScript and eventually alert the user to fill in these fields.

  We already prepare the form for these checks by including hints to show if needed. We'll style these hints later to be hidden by default.

```
<label for="email">Email*</label>
<span id="hint_email" class="hint"> - please enter an email address</span>
<input id="email" type="email" />

<label for="subject">Subject*</label>
<span id="hint_subject" class="hint"> - please enter a subject line</span>
<input id="subject" type="text" />

<label for="message">Message*</label>
<span id="hint_message" class="hint"> - please enter a message</span>
<textarea id="message"></textarea>
```

- For the next step you'll need a loading icon. Either create your own loading icon using one of the many online tools or download the one I use here. For our example, it should be called **loading.gif** and be saved in the same folder as our HTML page.

- The **loading** section is very simple. It just shows the loading icon:

```
<div id="loading">
    <img src="loading.gif" />
</div>
```

- The **success** and **error** sections are also quite simple:

```
<div id="success">
    <h2>Thank you!</h2>
    <p><a id="reset_success" href="#">Send another message</a></p>
</div>

<div id="error">
    <h2>Oops - something went wrong</h2>
    <p><a id="reset_error" href="#">Try again</a></p>
</div>
```

- Note the two links in these sections enabling users to send another message / try again. We'll add this functionality later in JavaScript.

- Finally, we edit **index.css** to style our page. We'll start with the **contact** section:

```
#contact {
    display: block;
}
fieldset {
    margin: 3em;
    padding: 2em;
}
input, textarea {
    display: block;
    margin-bottom: 1em;
    width: 100%;
}
textarea {
    height: 10em;
}
.hint {
    display: none;
    color: red;
}
```

- Note that elements with class **.hint** are hidden by default and will show in red

- Next the **loading** section:

```
#loading {
    display: none;
    text-align: center;
    padding-top: 9em;
}
```

- The **loading** section is hidden by default. It aligns the loading icon in the centre and has a large padding at the top to make sure the icon does not show at the top of the page but more towards the centre. *(Note this is a cheap hack - ideally we'd vertically align the icon in the middle. Maybe you can look into this later as a possible improvement...)*

- Next the **success** section - also hidden and centred and with a top padding to push content towards the middle of the page :

```
#success {
    display: none;
    text-align: center;
    padding-top: 10em;
}
```

- And finally the **error** section - also hidden and with a top padding:

```
#error {
    display: none;
    text-align: center;
    padding-top: 10em;
}

#error h2 {
    color: #ff0000;
}
```

- Note we also style the **<h2>** element in this section to show in red. This is meant to alert the user that something went wrong.

- Save all changes and refresh the page in the browser. You should see the form.

Step 3: Add JavaScript to submit the form

- After the page has loaded, select the **form** and add an event listener for the **submit** event:

```
window.addEventListener('load', function(){

    var form = document.querySelector('#contact form');
    form.addEventListener('submit', sendMessage);

});
```

- Note we reference an event listener function called **sendMessage**. This function does not exist yet, we'll create it in the following steps.

- Let's start by declaring the function. We declare it with a parameter **evt** (short for event) because we want to access functionality of the event object. In particular, we want to

prevent the default browser action for a form **submit** (which would be to automatically submit the form to the server and load the response as a new page):

```
function sendMessage(evt) {
    evt.preventDefault();

}
```

*(The next few code blocks go inside the **sentMessage** function, always at the end, before the closing curly bracket **}**. Don't forget to format the code in between if it does not align properly. In VSCode you can do this with the key combination **ALT+SHIFT+F**)*

- Now add code that gets the form field values and references to our hint messages:

```
// get values
var email = document.querySelector('#email').value.trim();
var subject = document.querySelector('#subject').value.trim();
var message = document.querySelector('#message').value.trim();


// get handles for hint messages
var hint_email = document.querySelector('#hint_email');
var hint_subject = document.querySelector('#hint_subject');
var hint_message = document.querySelector('#hint_message');
```

- Now add code that checks the form values and eventually shows the hint messages to alert the user.

  We use a variable called **fields_ok** for this part. It is initialised to **true** at the beginning of the checks and is set to **false** whenever there is a problem with any field value.

  If it is still **true** at the end, that means all field values passed the checks and we can proceed to submit them.

  Let's start with the **email** field. We check that it is at least 5 characters long (a@b.c) and contains an **@** sign (there could be more stringent checks, but this will do for now):

```
var fields_ok = true;

if(email.length < 5 || email.indexOf('@') < 0) {
    hint_email.style.display = 'inline';
    fields_ok = false;
} else {
    hint_email.style.display = 'none';
}
```

- Next the **subject** and **message** fields. We check that they are not empty (i.e. that their string length is not zero):

```
if(subject.length == 0) {
    hint_subject.style.display = 'inline';
    fields_ok = false;
} else {
    hint_subject.style.display = 'none';
}

if(message.length == 0) {
    hint_message.style.display = 'inline';
    fields_ok = false;
} else {
    hint_message.style.display = 'none';
}
```

- The next block of code is only executed if all the input fields are OK. It *simulates* submitting the form values to the server and in the process showing and hiding our **form / loading / success / error** sections to keep the user informed. (You can replace this later with the real thing in the Optional Step 4 if you like):

```
if(fields_ok) {

    // hide form and show loading icon
    document.querySelector('#contact').style.display = 'none';
    document.querySelector('#loading').style.display = 'block';

    // prepare data for transport to server
    var data = 'email=' + encodeURIComponent(email)
             + '&subject=' + encodeURIComponent(subject)
             + '&message=' + encodeURIComponent(message);

    // simulate submitting the data to the server via AJAX request
    setTimeout(function(){

         // this part executes once we get a (simulated) server response

        // hide loading icon when we receive a the response
        document.querySelector("#loading").style.display = "none";

       // simulated response: 75% chance of success
        var success = Math.random() > 0.25;

        // show success or error section depending on the response
        if(success) {
            document.querySelector("#success").style.display = "block";
        }
        else {
            document.querySelector("#error").style.display = "block";
        }

    }, 2000);
}
```

- Save all changes and try the form in the browser. It should submit, show a loading icon and then either a success message or an error message.

- What does not yet work at this stage is clicking the link in the success or error sections to *send another message* or *try again*. We'll add this functionality in the next step.

Step 4: Add JavaScript to reset the form

- Add event listeners to the links for users to *send another message* or *try again*. We need to do this all the way at the top in our page load event listener:

```
window.addEventListener('load', function(){

    var form = document.querySelector('#contact form');
    form.addEventListener('submit', sendMessage);

    var reset_success = document.querySelector("#reset_success");
    reset_success.addEventListener("click", reset);

    var reset_error = document.querySelector("#reset_error");
    reset_error.addEventListener("click", reset);

});
```

- Note that both links have the same event listener function called **`reset`**. This function does not exist yet, we'll create it next, all the way at the bottom of your JavaScript:

```
function reset(evt) {
    evt.preventDefault();
    document.querySelector('#success').style.display = 'none';
    document.querySelector('#error').style.display = 'none';
    document.querySelector('#loading').style.display = 'none';
    document.querySelector('#contact').style.display = 'block';
    document.querySelector('#email').value = '';
    document.querySelector('#subject').value = '';
    document.querySelector('#message').value = '';
}
```

- This function is quite simple. It prevents the default action for links (which would be to go to the link target) and then selects page elements and resets them to their default state: hiding all sections apart from the contact form and setting all input fields to empty (`''`).

- Save all changes and try the form and rest links in the browser.

- Done

Optional Step 5: Submit form values to server

- In this step we'll modify the JavaScript to make an actual AJAX request to the server. We then upload everything to brighton.domains to make it work online. This also will involve uploading a PHP server script and setting its file permissions so that it can be executed.

- Let's start with the JavaScript code. Inside the **sendMessage** function, replace the simulated form submit with the real one:

```
if(fields_ok) {

    // hide form and show loading icon
    document.querySelector('#contact').style.display = 'none';
    document.querySelector('#loading').style.display = 'block';

    // prepare data for transport to server
    var data = 'email=' + encodeURIComponent(email)
             + '&subject=' + encodeURIComponent(subject)
             + '&message=' + encodeURIComponent(message);

    // prepare the server request
    var xhr = new XMLHttpRequest();
    xhr.addEventListener('load', function(){

        // hide loading icon
        document.querySelector('#loading').style.display = 'none';

        // check response and show either success or error sections
        if(xhr.status==201){
            document.querySelector('#success').style.display = 'block';
        }
        else {
            document.querySelector('#error').style.display = 'block';
        }
    });

    // send data to a server script called contact.php
    xhr.open('POST', 'contact.php', true);
    xhr.setRequestHeader('Content-type', 'application/x-www-form-urlencoded');
    xhr.send(data);
}
```

- Save the changes. Look at the 3rd line from the bottom specifying **contact.php** as the server script receiving the data. There are several things to note:
  - **contact.php** is located in the same folder as the index.html page
  - **contact.php** must run on the server, it cannot run from our local file system
  - that means we have to upload everything to brighton.domains in order to test it
  - for server security, we must set appropriate file permissions for **contact.php** to run

- To get started, download the server script here. Save it to the same folder as your contact form, then rename it from **contact.php.txt** to **contact.php**.

(The reason why it downloads as **`contact.php.txt`** is that the server executes files ending in **`.php`** and returns the results, making it impossible to download the **`.php`** script itself. This is a good thing as server scripts often contain sensitive information like database names and passwords. To get around this and enable downloading the script, I added a **`.txt`** file extension that tricks the server into not executing it but simply returning the script as text.)

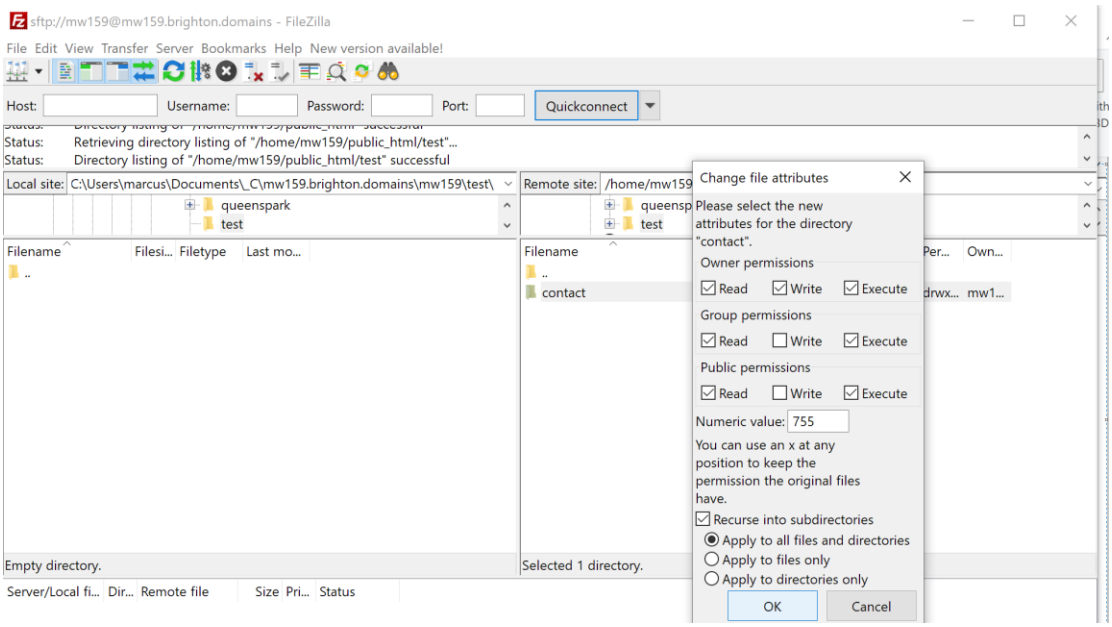
- Next we upload **`index.html, index.css, index.php, loading.gif`** and **`contact.php`** to a new folder in our web space on brighton.domains.

  It's a good idea to use an FTP client like FileZilla for this, however, you also can do it via the File Manager web interface in the brighton.domains admin dashboard. If you use an FTP client, you can find your FTP login details in your dashboard under Manage Your Account:

  (You might need to set an FTP password there if you haven't done so before. This should be different from your uni password!)

- Once uploaded, you need to set appropriate file permissions for **`contact.php`** to execute. The folder containing the script will need the same permissions.

  In the FileZilla FTP client, right-click the **`folder`** on your server containing **`contact.php`** and give it permissions **`755`**. Check the box *"Recurse into subdirectories"* and select *"Apply to all files and directories"*, then click OK.



- Done. Call up the online version of your contact form in the browser and test it!